



同濟大學
TONGJI UNIVERSITY

数据库系统设计中期报告

课题名称 台风实时监测及历史查询系统

学 院 计算机科学与技术学院

组员及分工 2354366 徐华炫 需求分析、概念设计

2352841 刘佳鑫 逻辑设计

2351520 毛星博 物理设计、功能优化

日 期 2025-06-07

1 需求分析

1.1 研究背景及意义

2018 年，台风“山竹”横扫菲律宾吕宋后登陆广东，香港测得破纪录风暴潮（大澳水位 3.9 米），粤港澳经济损失超 50 亿美元；2023 年，台风“杜苏芮”登陆中国东南沿海，为近 74 年来闽南最强台风，福建、浙江超 416 万人受灾，经济损失超 150 亿元。台风作为频繁侵袭沿海区域的重大气象灾害，对民众生命和社会经济有着巨大的威胁，因此，构建一套完善的台风监测预警系统具有关键性战略意义。

纵观网上的各种台风实时监测平台，我们发现这些平台大多只有简单的路径预测，用户很难由此清晰地认识到某个区域所面临的危险程度。所以我们小组期望开发这样一个系统：能对过往台风登陆地点及损害情况作分析，提供给用户中国沿海各区域的台风受灾程度分布图，并结合台风实时路径监测系统，给那些可能面临较大危险区域的用户发送不同等级的预警，更好地达到台风监测平台防灾、避灾的目的。

1.2 功能需求

数据库基础功能点包括多源数据存储、系统前段可视化查询、用户预警等。

1.2.1 多源数据存储

多源数据存储上，数据库设计包含过去二十年的台风的登陆信息以及各登陆地的受灾情况、当前台风的路径监测数据等。需收录近二十年所有在我国沿海登陆的台风信息，具体包括：台风基础属性：国际编号、中文命名、起止时间，登陆点精确记录：至少包含登陆时间、中心位置、登陆强度（中心风速/气压），灾情损失档案：按县级行政单元统计人员伤亡、房屋损毁、经济损失等量化指标。实时监测数据上，包含各时刻的路径节点数据：台风中心经纬度、移动方向、移动速度、中心气压、最大风速等。

1.2.2 可视化查询

可视化查询方面，支持灾情热力图生成，可定位至县级行政单元的灾情渲染，支持按时间筛选过滤，受灾指标包括经济损失/伤亡人数/房屋损毁率等。支持台风路径可视化，绘制台风各路径节点，提供可视化的台风路径，并结合轨迹预测，不同强度的台风要有不同的视觉呈现效果。

1.2.3 用户预警功能

用户预警功能需求上，计算受灾概率：实时台风路径 20 公里缓冲区 + 历史相似案例匹配。分级预警规则：结合受灾概率、登陆情况下受灾的严重程度的预测(根据过往数据)进行分析，分为蓝色：基础防范提示，黄色：避险方案推送，橙色：应急资源预置，红色：强制撤离指令。

1.3 可行性分析

1.3.1 数据来源

数据来源上，我们找到了从 1949 年至 2024 年的台风信息：<https://cndeeptdata.pro/2024/09/16/cn-dd-0243-1949-2024%E5%B9%B4%E4%B8%AD%E5%9B%BD%E5%8F%B0%E9%A3%8E%E4%BA%8B%E4%BB%B6%E7%BB%9F%E8%AE%A1/>，结合网络各地受灾情况报道，可以构建出历史台风数据。

<https://typhoon.slt.zj.gov.cn/> 网址可以查看实时的台风信息，导入数据库可呈现台风路径并与历史数据分析结合，给用户预警。

1.3.2 实现可行性

系统拟采用 MySQL 8.0 作为核心数据库，配合前端三件套及 Python 后端技术栈进行开发实现。从技术实现层面看，热力图和台风路径等可视化、用户系统的开发不难实现，关键在于历史灾情数据的构建，考虑到我们获取到了详细的台风登陆信息及受灾情况，故将数据导入数据库后，数据能得到较好的整合，系统总体实现可行性得到了有效保障。

2 概念设计

2.1 实体设计

2.1.1 台风基本信息

该表中存储着每个台风的全局唯一标识信息，包括台风编号、中英文名称。同时，还记录了台风的生成时间和消散时间，以实现对台风生命周期的完整追踪。台风的基本信息是台风登陆记录、灾害损失记录、台风路径监测等表的基础，记录了台风的基础信息。



2.1.2 台风登陆记录



该表中存储着每次台风登陆的详细空间位置信息，包括精确到乡镇级的登陆位置坐标（经纬度）。同时，还记录了登陆时的风速、气压等关键气象参数，以实现对登陆强度的量化分析。此外，表中还记录了登陆时间及行政区划信息，从而为区域灾害评估提供空间位置基础。

2.1.3 灾害损失记录

该表中存储着台风在各县级行政单元造成的灾害损失数据，包括人员伤亡、房屋损毁等核心灾情指标。同时，还记录了经济损失和最大降雨量等量化数据，以实现灾害影响的综合评估。此外，表中还建立了与台风编号的关联关系，从而为历史灾情分析提供结构化数据支持。



2.1.4 台风路径监测

该表中存储着台风生命周期内的动态路径点数据，包括每个时间更新的位置坐标及移动参数。同时，还记录了实时风速、气压变化等关键指标，以实现台风强度的连续监测。此外，表中还标注了预测点与实测点的类型区分，从而区分记录点和预测点。



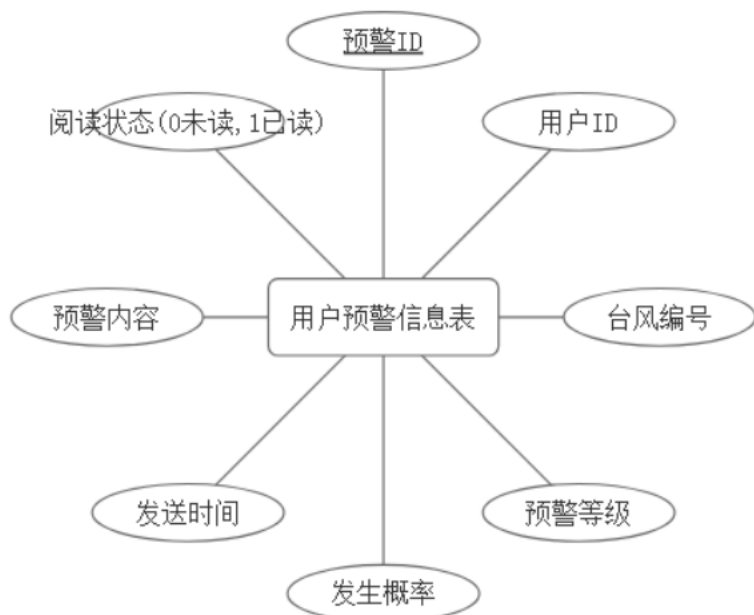
```
graph TD; A([规则ID]) --- B[预警规则表]; C([应对措施]) --- B; D([预警等级名称]) --- B; E([概率上限]) --- B; F([概率下限]) --- B;
```

2.1.6 用户信息

```
graph TD; A([用户ID]) --- B[用户信息表]; C([用户名]) --- B; D([密码]) --- B; E([手机号]) --- B; F([省份]) --- B; G([城市]) --- B; H([区县]) --- B; I([详细地址]) --- B; J([电子邮箱]) --- B; K([创建时间]) --- B; L([最后登录时间]) --- B;
```

2.1.7 用户预警信息

该表中存储着每位用户接收到的实时预警信息，包括预警级别和精确受灾概率。同时，还记录了预警内容推送时间和阅读状态，以实现预警闭环管理。



2.2 实体联系设计

2.2.1 台风基础信息表和各表的联系

在台风数据库设计中，台风基础信息与其他多个核心数据表之间存在一对多层级关联：

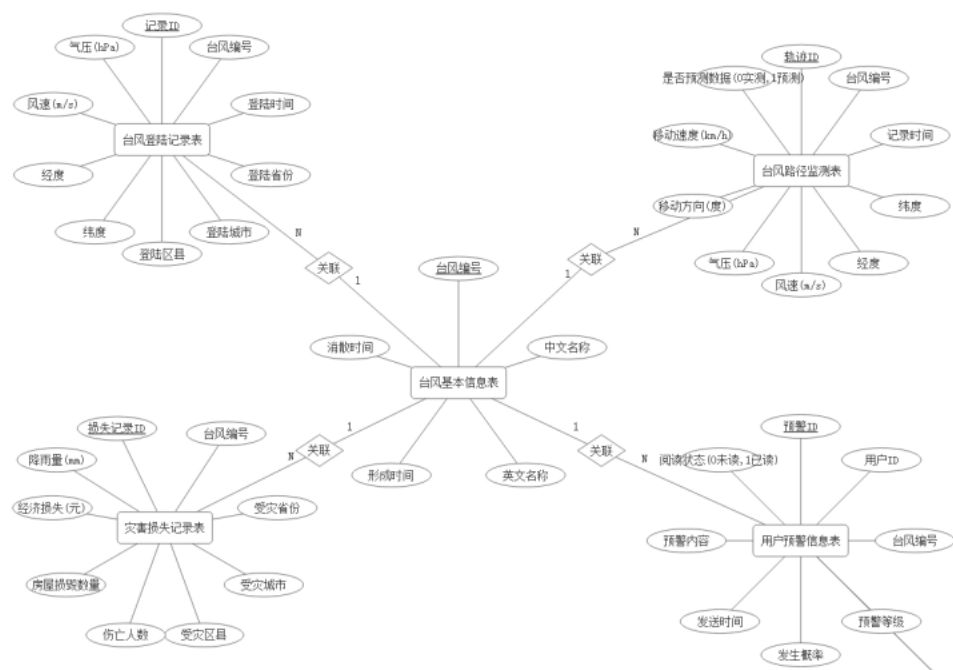
台风与登陆记录表：一个台风可能包含多次登陆事件（如 2023 年台风"杜苏芮"先后登陆 3 次），每次登陆都会在 `landfall_record` 表中产生独立记录，形成"台风-登陆事件"的树状关系。

台风与灾情数据表：同一台风会在不同地区引发灾害（如 2019 年"利奇马"影响 6 省 58 县），通过 `damage_record` 表按县级单位建立辐射状灾害记录。

台风与路径监测表：在台风生命周期中会产生平均 200+个路径点，`track_monitoring` 表以 10 分钟为颗粒度构建时空数据链条。

用户预警信息表：同一个台风会产生多个发送给各个用户的预警信息。

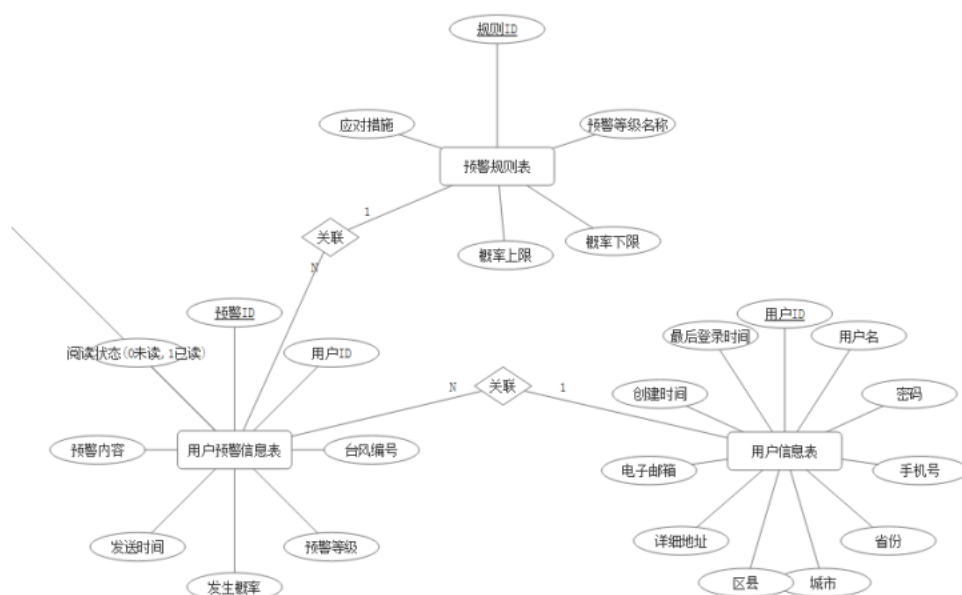
此外，像台风登陆记录表的城市、区县等也和灾害损失记录表中的受灾地有联系。



2.2.2 用户预警信息表和各表的联系

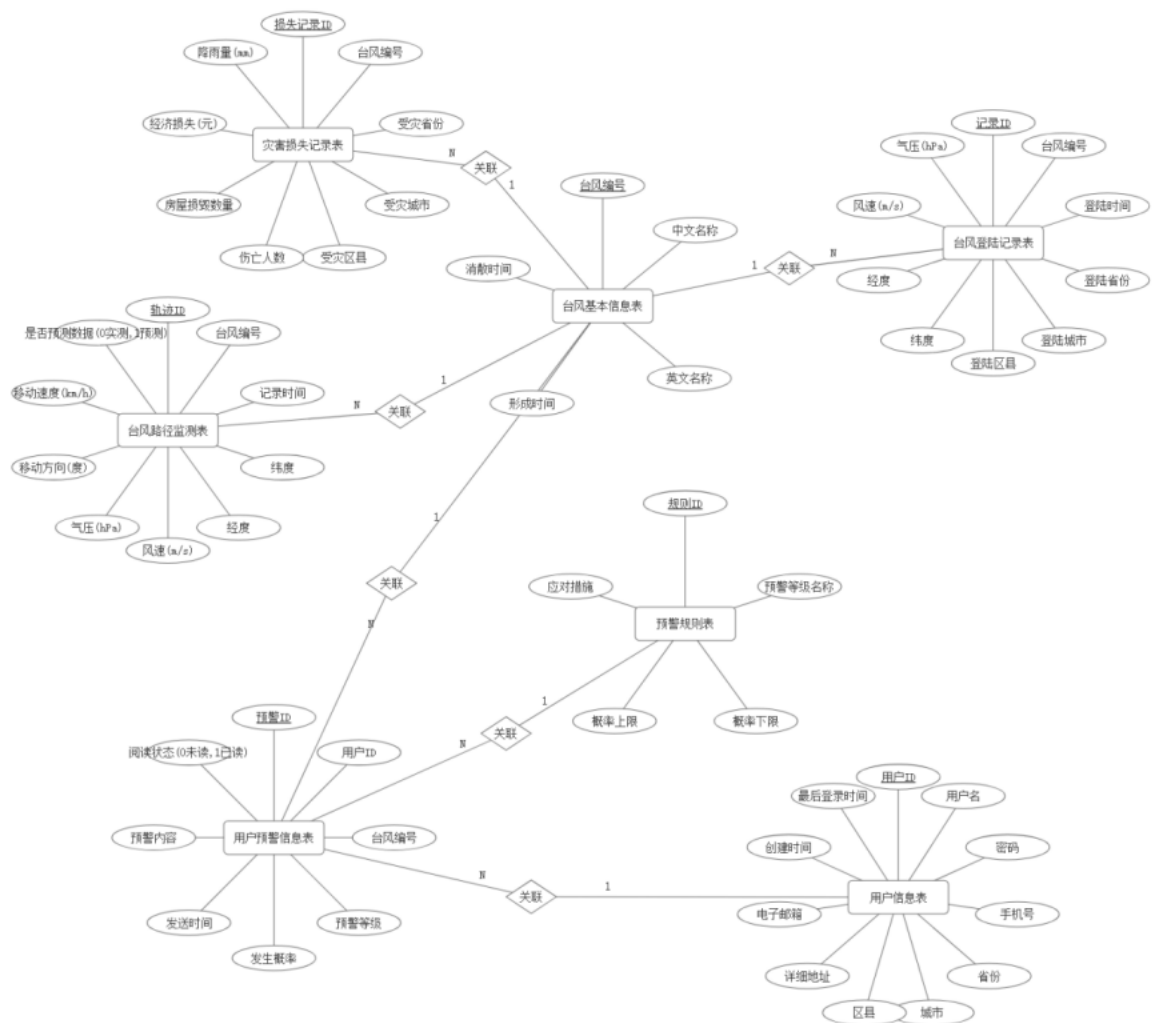
同一个预警等级会对应多个具体的预警实例，同理同一个用户也对应着多个预警信息。

此外，用户预警动态关联机制上，用户系统与灾害数据构成多维度交叉关联：每个用户在 user_info 中绑定省市三级位置信息，根据登陆记录表中的经纬度坐标与用户距离计算判断是否可能受灾，灾情记录表 (damage_record) 构建区域风险热力模型，再根据这些信息判断生成危险程度，根据 warning_rule 阈值触发分级预警。



2.3 总 E-R 图设计

最后是总的 E-R 图：



3 逻辑设计

3.1 实体设计

进行如下的实体间联系转换：

对于 1 对 1 联系：在任意一方中加入另一方的主码并设为其外码，并加入联系本身的属性；
对于 1 对 n 联系：将 1 方的主码加入 n 方作为外码，同时将联系的属性加入 n 方；
对于 n 对 n 联系：将联系本身转换为一个关系模式，将联系双方的主码加入其中设为码，并将联系的属性也加入其中。

设计出的关系 E-R 图可以转换为如下关系模型：

台风基本信息实体 Typhoon_base (typhoon_id, name_cn, name_en, start_time, end_time);

台风登陆信息实体 Landfall_record (record_id, typhoon_id, landfall_time, province, city, district, latitude, longitude, speed, pressure);

typhoon_id REFERENCES typhoon_base(*typhoon_id*)

受灾情况统计实体 Damage_record (damage_id, typhoon_id, province, city, district, casualties, houses_damaged, economic_loss, rainfall);

typhoon_id REFERENCES typhoon_base(*typhoon_id*)

台风路径监测实体 Track_monitoring (track_id, typhoon_id, record_time, latitude, longitude, wind_speed, pressure, move_direction, move_speed, is_forecast);

typhoon_id REFERENCES typhoon_base(*typhoon_id*)

预警规则实体 Warning_rule (rule_id, level_name, probability_min, action, probability_max);

用户信息实体 User_info (user_id, username, password, phone, province, city, district, address, email, create_time, last_login);

用户预警记录实体 User_warning (warning_id, user_id, typhoon_id, rule_id, probability, send_time, content, read_status);

user_id REFERENCES user_info(*user_id*),

typhoon_id REFERENCES typhoon_base(*typhoon_id*),

rule_id REFERENCES warning_rule(*rule_id*)

本 E-R 图内的所有关联关系均为 1 对 N 关系，在所有的 N 方实体中都添加了 1 应的主键以

保留实体间的联系信息。

本关系模型同样满足 3NF 范式分解，3NF 范式要求关系中的每一列都必须直接依赖于主键。既不能有部分依赖，即非主属性依赖于部分的主属性；也不能有传递依赖，即更低级别的非主属性依赖于更高级别的非主属性。

在上述关系模型中，所有实体只有一个主键。实体中的每一列都直接依赖于所有的主键，没有出现部分依赖的情况。例如，在普通用户实体中，用户姓名、密码、性别、联系方式、所在城区、邮箱以及上次登录时间都直接依赖于用户编号这一主键，不存在其他属性的间接依赖关系。

此外，在上述关系模型中也没有出现更低级别的非主属性依赖于更高级别的非主属性，即传递依赖的情况。

3.2 实体表设计

3.2.1 台风基本信息表

属性名	说明	数据类型	默认值	约束
typhon_id	台风 ID	VARCHAR(10)	\	主键
name_cn	中文名	VARCHAR(20)	\	\
name_en	英文名	VARCHAR(30)	\	\
start_time	形成时间	DATETIME	\	非空
end_time	消散时间	DATETIME	\	\

3.2.2 台风登陆信息表

属性名	说明	数据类型	默认值	约束
record_id	记录 ID	INT	\	主键，自增
typhon_id	台风 ID	VARCHAR(10)	\	外键
landfall_time	登录时间	DATETIME	\	非空
province	登陆省份	VARCHAR(20)	\	非空
city	登陆城市	VARCHAR(20)	\	非空
district	登陆区县	VARCHAR(20)	\	非空
latitude	纬度	DECIMAL(9,6)	\	非空
longitude	经度	DECIMAL(9,6)	\	非空
speed	风速(m/s)	DECIMAL(6,1)	\	非空
pressure	气压(pha)	SMALLINT	\	非空

3.2.3 受灾情况统计表

属性名	说明	数据类型	默认值	约束
damage_id	记录 ID	INT	\	主键，自增
typhoon_id	台风 ID	VARCHAR(10)	\	外键
province	受灾省份	VARCHAR(20)	\	非空
city	受灾城市	VARCHAR(20)	\	非空
district	受灾区县	VARCHAR(20)	\	非空
casualties	伤亡人数	INT	0	\
houses_damaged	受损房屋	INT	0	\
economic_loss	经济损失	BIGINT	0	\
rainfall	降雨量	DECIMAL(6,1)	\	\

3.2.4 台风路径监测信息表

属性名	说明	数据类型	默认值	约束
track_id	记录 ID	INT	\	主键，自增
typhoon_id	台风 ID	VARCHAR(10)	\	外键
record_time	记录时间	DATETIME	\	非空
latitude	纬度	DECIMAL(9,6)	\	非空
longitude	经度	DECIMAL(9,6)	\	非空
wind_speed	风速(m/s)	DECIMAL(6,1)	\	非空
pressure	压强(hpa)	SMALLINT	\	非空
move_direction	移动方向(度)	SMALLINT	\	\
move_speed	移速(km/h)	DECIMAL(5,1)	\	\
is_forecast	是否为预测数据	TINYINT(1)	0	\

3.2.5 预警规则表

属性名	说明	数据类型	默认值	约束
rule_id	规则 ID	INT	\	主键，自增
level_name	预警等级	VARCHAR(10)	\	非空
probability_min	概率上限	DECIMAL(3,2)	\	非空
probability_max	概率下限	DECIMAL(3,2)	\	非空
action	应对措施	TEXT	\	非空

3.2.6 用户信息表

属性名	说明	数据类型	默认值	约束
user_id	用户 ID	INT	\	主键，自增
username	用户名	VARCHAR(50)	\	非空，非重
password	密码	VARCHAR(128)	\	非空
phone	联系电话	VARCHAR(20)	\	非空，非重
province	居住省份	VARCHAR(20)	\	非空
city	居住城市	VARCHAR(20)	\	非空
district	所在区县	VARCHAR(20)	\	非空
address	详细地址	VARCHAR(100)	\	\
email	邮箱	VARCHAR(50)	\	\
create_time	账户建立时间	DATETIME	当前时间	\
last_login	上次登录时间	DATETIME	\	\

3.2.7 用户预警记录表

属性名	说明	数据类型	默认值	约束
warning_id	记录 ID	INT	\	主键，自增
user_id	用户 ID	INT	\	外键
typhoon_id	台风编号	VARCHAR(10)	\	外键
rule_id	预警规则 ID	TINYINT	\	外键
probability	发生概率	DECIMAL(4,3)	\	非空
send_time	发送时间	DATETIME	当前时间	\
content	预警内容	TEXT	\	非空
read_status	阅读状态	TINYINT(1)	0	\

4 物理设计

数据库物理设计的目标是在具体的硬件环境和数据库管理系统（DBMS）上，为逻辑数据模型选择最合适的物理结构，包括存储结构、存取方法等，以获得最佳的系统性能和资源利用率。本项目选用 MySQL 8.0 作为数据库管理系统，以下设计均基于此环境。

4.1 存储引擎选择

本项目全部数据表统一选用 InnoDB 存储引擎。

选择理由：

① 支持事务（ACID 特性）：系统涉及用户注册、预警信息生成与读取等操作，需要事务来保证数据的一致性和完整性。例如，生成一条预警信息并更新用户状态应作为一个原子操作完成。

② 支持外键约束：项目逻辑设计中定义了大量的表间关联，如 User_warning 表通过外键关联了用户、台风和预警规则表。InnoDB 提供了完整的申明和强制外键约束的能力，能从数据层面保证引用的完整性。

③ 行级锁定（Row-Level Locking）：本系统未来可能面临高并发的写入场景，例如台风来临时，系统需要同时向大量用户写入预警信息。InnoDB 的行级锁机制只锁定被修改的行，而不是整个表，这极大地提升了系统在高并发读写下的性能。

④ 崩溃恢复能力：InnoDB 提供了强大的崩溃恢复能力，能确保在服务器宕机等意外情况下，数据库能恢复到崩溃前的状态，保障数据安全。

SQL 实现：

```
ALTER TABLE Typhoon_base ENGINE=InnoDB;
ALTER TABLE Landfall_record ENGINE=InnoDB;
ALTER TABLE Damage_record ENGINE=InnoDB;
ALTER TABLE Track_monitoring ENGINE=InnoDB;
ALTER TABLE Warning_rule ENGINE=InnoDB;
ALTER TABLE User_info ENGINE=InnoDB;
ALTER TABLE User_warning ENGINE=InnoDB;
```

4.2 索引设计

4.2.1 主键索引

主键是表中每一行数据的唯一标识符。为表定义主键后，MySQL 会自动为其创建唯一的主键索引（PRIMARY Index）。主键索引不仅能保证数据的唯一性，还能极大地提升基于主键的查询速度，并且 InnoDB 表本身就是围绕主键索引构建的索引组织表。

根据逻辑设计，各表均已定义主键。

Typhoon_base 表的主键为 typhoon_id

Landfall_record 表的主键为 record_id

Damage_record 表的主键为 damage_id

Track_monitoring 表的主键为 track_id

Warning_rule 表的主键为 rule_id

User_info 表的主键为 user_id

User_warning 表的主键为 warning_id

4.2.2 外键索引

外键用于建立和加强两个表数据之间的链接。为外键列创建索引至关重要。第一，在进行表连接（JOIN）查询时，如果外键没有索引，数据库将对关联表进行全表扫描，效率极低；第二，在对父表进行更新或删除操作时，如果子表的外键列没有索引，会导致对父表产生长时间的锁定，影响并发性能。

字段选择与 SQL 实现：

```
-- Landfall_record 表中的外键 typhoon_id
CREATE INDEX idx_landfall_typhoon_id ON Landfall_record(typhoon_id);

-- Damage_record 表中的外键 typhoon_id
CREATE INDEX idx_damage_typhoon_id ON Damage_record(typhoon_id);
```



```
-- Track_monitoring 表中的外键 typhoon_id
CREATE INDEX idx_track_typhoon_id ON Track_monitoring(typhoon_id);

-- User_warning 表中的外键 user_id, typhoon_id, rule_id
CREATE INDEX idx_userwarn_user_id ON User_warning(user_id);
CREATE INDEX idx_userwarn_typhoon_id ON User_warning(typhoon_id);
CREATE INDEX idx_userwarn_rule_id ON User_warning(rule_id);
```

4.2.3 单列索引

为经常作为查询条件（出现在 `where` 子句中）、排序（`order by`）或分组（`group by`）依据的单个列创建索引，可以显著提高查询性能。特别是对于具有高区分度的列（如用户手机号），创建唯一索引还能在数据库层面保证其值的唯一性。

SQL 实现：

用户登录与唯一性校验：User_info 表的 `username` 和 `phone` 字段是用户登录和身份识别的关键，且要求不能重复。

```
CREATE UNIQUE INDEX uk_user_username ON User_info(username);
CREATE UNIQUE INDEX uk_user_phone ON User_info(phone);
```

按时间查询台风：Typhoon_base 表的 `start_time` 字段可用于按时间范围筛选台风。

```
CREATE INDEX idx_typhoon_start_time ON Typhoon_base(start_time);
```

4.2.4 组合索引

当查询条件经常涉及多个列时，创建组合索引（覆盖多个列的索引）通常比为每个列创建单列索引更有效。组合索引遵循“最左前缀原则”，查询时从索引的最左列开始匹配。

SQL 实现：

台风路径查询：Track_monitoring 表的核心查询场景是获取指定台风的路径，通常会同时使用 `typhoon_id` 和 `record_time` 进行筛选和排序。

```
CREATE INDEX idx_track_typhoon_time ON Track_monitoring(typhoon_id, record_time);
```

灾情地区查询：Damage_record 表的核心查询是按行政区划（省、市、县）检索灾情信息。

```
CREATE INDEX idx_damage_area ON Damage_record(province, city, district);
```

4.2.5 空间索引

本系统核心功能之一是“实时台风路径 20 公里缓冲区 + 历史相似案例匹配”来计算受灾概率并发起预警。这属于典型的地理空间邻近查询（Proximity Search）。传统的 B-Tree 索引不适用于处理“附近”这类二维空间查询。MySQL 的空间索引（SPATIAL Index）使用 R-Tree 数据结构，专门用于高效处理地理空间数据查询。

SQL 实现：我们需要对存储了经纬度信息的 User_info、Landfall_record 和 Track_monitoring 表进行改造，并创建空间索引。

- ① 为 User_info 表添加空间索引（用于查找受影响用户）：

```
ALTER TABLE User_info ADD COLUMN location POINT NOT NULL SRID 4326;  
UPDATE User_info SET location = POINT(longitude, latitude);  
CREATE SPATIAL INDEX sp_idx_user_location ON User_info(location);
```

- ② 为 Landfall_record 表添加空间索引（用于分析登陆点）：

```
ALTER TABLE Landfall_record ADD COLUMN location POINT NOT NULL SRID 4326;  
UPDATE Landfall_record SET location = POINT(longitude, latitude);  
CREATE SPATIAL INDEX sp_idx_landfall_location ON Landfall_record(location);
```

- ③ 为 Track_monitoring 表添加空间索引（用于分析路径点）：

```
ALTER TABLE Track_monitoring ADD COLUMN location POINT NOT NULL SRID 4326;  
UPDATE Track_monitoring SET location = POINT(longitude, latitude);  
CREATE SPATIAL INDEX sp_idx_track_location ON Track_monitoring(location);
```

4.2.6 索引设计汇总表

表名	属性	索引类型	描述
Typhoon_base	typhoon_id	主键索引	台风信息的唯一标识，是所有查询的基础。
	start_time	普通索引	便于按时间范围快速筛选台风。
Landfall_record	record_id	主键索引	台风登陆记录的唯一标识。
	typhoon_id	普通索引 (外键)	作为外键，加速与 Typhoon_base 表的连接查询。
	location	空间索引	用于高效查询特定地理位置附近的台风登陆点。
Damage_record	damage_id	主键索引	灾情记录的唯一标识。
	typhoon_id	普通索引 (外键)	作为外键，加速与 Typhoon_base 表的连接查询。
	(province, city, district)	组合索引	核心业务索引，用于按行政区划快速统计和查询灾情数据，为热力图生成提供支持。
Track_monitoring	track_id	主键索引	台风路径点的唯一标识，数据量极大。
	(typhoon_id, record_time)	组合索引	核心业务索引，高效查询特定台风的完整或部分时间段内的路径轨迹。
	location	空间索引	核心业务索引，用于高效计算路径点附近的地理目标，是实现用户预警的关键。
Warning_rule	rule_id	主键索引	预警规则的唯一标识。
User_info	user_id	主键索引	用户的唯一标识。

	username	唯一索引	加速用户登录验证，并保证用户名的唯一性。
	phone	唯一索引	加速手机号查找，并保证联系方式的唯一性。
	location	空间索引	核心业务索引，用于根据台风路径快速匹配受影响区域内的用户。
User_warning	warning_id	主键索引	用户预警记录的唯一标识。
	user_id	普通索引 (外键)	作为外键，加速查询特定用户收到的所有预警信息。
	typhoon_id	普通索引 (外键)	作为外键，加速查询由特定台风触发的所有预警信息。
	rule_id	普通索引 (外键)	作为外键，加速与 Warning_rule 表的连接。

4.3 分区设计

对于 **Track_monitoring** 和 **Damage_record** 这种将随时间无限增长的“流水”表，当数据量达到千万甚至上亿级别时，单表的查询和维护（如删除历史数据）性能会急剧下降。通过分区技术，可以将一个大表在物理上分割成多个更小、更易于管理的部分，但逻辑上仍作为一张表来操作。分区能极大提升涉及分区键的查询效率，并能通过删除整个分区的方式实现历史数据的快速归档和清理。

SQL 实现：

我们选择按时间对 **Track_monitoring** 表和 **Landfall_record** 表进行范围分区。

- ① 对 **Track_monitoring** 表按 **record_time** 的年份进行分区：

```
ALTER TABLE Track_monitoring PARTITION BY RANGE (YEAR(record_time)) (
    PARTITION p_before_2020 VALUES LESS THAN (2020),
```

```
PARTITION p2020 VALUES LESS THAN (2021),
PARTITION p2021 VALUES LESS THAN (2022),
PARTITION p2022 VALUES LESS THAN (2023),
PARTITION p2023 VALUES LESS THAN (2024),
PARTITION p2024 VALUES LESS THAN (2025),
PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

② 对 Landfall_record 表按 landfall_time 的年份进行分区：

```
ALTER TABLE Landfall_record PARTITION BY RANGE (YEAR(landfall_time)) (
PARTITION p_before_2020 VALUES LESS THAN (2020),
PARTITION p2020 VALUES LESS THAN (2021),
PARTITION p2021 VALUES LESS THAN (2022),
PARTITION p2022 VALUES LESS THAN (2023),
PARTITION p2023 VALUES LESS THAN (2024),
PARTITION p2024 VALUES LESS THAN (2025),
PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

5 功能优化

5.1 视图

视图可以简化复杂的查询逻辑、限制数据访问和保证逻辑数据的独立性。对于前端应用开发，可以直接查询预先定义好的视图，而无需编写复杂的 JOIN 和聚合语句。

SQL 实现：

① 创建 V_Active_Typhoon_Latest_Info 视图：用于实时获取所有活动台风（未消散）的最新位置、强度等信息。

```
CREATE OR REPLACE VIEW V_Active_Typhoon_Latest_Info AS
SELECT
    tb.typhoon_id,
    tb.name_cn,
    tb.name_en,
    tm.record_time,
    tm.latitude,
    tm.longitude,
    tm.wind_speed,
    tm.pressure,
    tm.move_direction,
    tm.move_speed
FROM
    Typhoon_base tb
JOIN
```

```
Track_monitoring tm ON tb.typhoon_id = tm.typhoon_id
JOIN
(SELECT typhoon_id, MAX(record_time) AS max_time FROM Track_monitoring GROUP BY
typhoon_id) AS latest
ON tm.typhoon_id = latest.typhoon_id AND tm.record_time = latest.max_time
WHERE
tb.end_time IS NULL;
```

② 创建 V_District_Damage_Summary 视图：用于按区县统计总灾情，为生成灾情热力图提供数据源。

```
CREATE OR REPLACE VIEW V_District_Damage_Summary AS
SELECT
    province,
    city,
    district,
    COUNT(DISTINCT typhoon_id) AS typhoon_count,
    SUM(casualties) AS total_casualties,
    SUM(houses_damaged) AS total_houses_damaged,
    SUM(economic_loss) AS total_economic_loss,
    AVG(rainfall) AS avg_rainfall
FROM
    Damage_record
GROUP BY
    province, city, district;
```

5.2 触发器

TRG_UserInfo_Before_Update_Set_LastLogin：在用户信息被更新前，自动更新其 last_login 字段为当前时间。

```
DELIMITER $$
CREATE TRIGGER TRG_UserInfo_Before_Update_Set_LastLogin
BEFORE UPDATE ON User_info
FOR EACH ROW
BEGIN
    SET NEW.last_login = NOW();
END;$$
DELIMITER ;
```

TRG_TyphoonBase_After_Insert_Log：当一个新的台风基础信息被插入时，自动在日志表中记录一条信息。

```
-- 创建一个日志表
CREATE TABLE Audit_Log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    log_time DATETIME NOT NULL,
    log_message VARCHAR(255) NOT NULL
);

-- 创建触发器
DELIMITER $$
CREATE TRIGGER TRG_TyphoonBase_After_Insert_Log
AFTER INSERT ON Typhoon_base
```

```
FOR EACH ROW
BEGIN
    INSERT INTO Audit_Log (log_time, log_message)
    VALUES (NOW(), CONCAT('New typhoon recorded: ', NEW.typhoon_id, ' (', NEW.name_cn, ')'));
END;$$
DELIMITER ;
```

5.3 安全性和权限管理

为了保护数据库免受未经授权访问和恶意操作，需要实施严格的权限管理策略。基本原则是“最小权限原则”，即只授予用户执行其工作所必需的最小权限。

SQL 实现：

我们创建三种不同的数据库用户角色：应用服务角色、数据管理员角色和只读分析角色。

创建用户：

```
CREATE USER 'app_server'@'localhost' IDENTIFIED BY 'AppServerPassword123';
CREATE USER 'data_admin'@'localhost' IDENTIFIED BY 'DataAdminPassword123';
CREATE USER 'readonly_user'@'localhost' IDENTIFIED BY 'ReadOnlyPassword123';
```

为 app_server 角色授权（Web 后端应用使用的账户）：

```
-- 授予查询权限
GRANT SELECT ON TyphoonDB.V_Active_Typhoon_Latest_Info TO 'app_server'@'localhost';
GRANT SELECT ON TyphoonDB.V_District_Damage_Summary TO 'app_server'@'localhost';
GRANT SELECT ON TyphoonDB.Typhoon_base, TyphoonDB.Landfall_record, TyphoonDB.Damage_record,
TyphoonDB.Track_monitoring, TyphoonDB.Warning_rule TO 'app_server'@'localhost';

-- 授予对用户信息和预警信息的增删改权限
GRANT SELECT, INSERT, UPDATE ON TyphoonDB.User_info TO 'app_server'@'localhost';
GRANT SELECT, INSERT, UPDATE ON TyphoonDB.User_warning TO 'app_server'@'localhost';

-- 授予执行特定存储过程的权限
GRANT EXECUTE ON PROCEDURE TyphoonDB.SP_Generate_Warnings_For_Typhoon TO
'app_server'@'localhost';
```

为 data_admin 角色授权（数据维护人员使用的账户）：

```
GRANT SELECT, INSERT, UPDATE, DELETE ON TyphoonDB.* TO 'data_admin'@'localhost';
```

为 readonly_user 角色授权（数据分析人员使用的账户）：

```
GRANT SELECT ON TyphoonDB.* TO 'readonly_user'@'localhost';
```